

# A Lightweight NMEA 0183 Parser for Field Validation of a Wearable GNSS Subsystem

Huy Hoang Le

Technical Lab Assistant (Software & Hardware) at  
EDABK Lab, Hanoi University of Science and Technology (HUST)  
Hanoi, Vietnam

**Abstract**—This report presents the design and field evaluation of a lightweight, dependency-free NMEA 0183 parser developed to support validation of the GNSS subsystem of a Screenless Smart Wristband under development at EDABK Lab, Hanoi University of Science and Technology. The wristband, built around a Nordic nRF52840 microcontroller and a Quectel LC76G multi-constellation GNSS module, has no display, so raw GNSS output is the primary signal available for debugging position and fix quality during field testing. This output is dense, abbreviation-heavy, and interleaved across up to five satellite constellations, which makes manual inspection slow and error-prone. The parser described here takes in timestamped raw log files, isolates and tokenizes individual NMEA sentences, converts coordinates from the NMEA degree-minute format to signed decimal degrees, and then finally produces a structured, per-sentence-type, human-readable report together with running statistics.

I describe the parsing pipeline, the dispatch logic for six core sentence types (GGA, RMC, GSA, GSV, VTG, GLL), and the program's behavior on malformed or incomplete input. Evaluation on a 131-second field log containing 1300 parsed NMEA sentences across four GNSS constellations confirms correct decoding of position, satellite-count, and fix-quality fields. It demonstrates good handling of non-NMEA content (which includes a dropped-byte fragment and an unplanned mid-capture reboot of the GNSS module) without crashing or producing corrupted output. My program is intended as a low-overhead diagnostic aid for ongoing GNSS validation work on the wristband project.

**Index Terms**—NMEA 0183, GNSS, embedded systems, log parsing, wearable devices, multi-constellation positioning

## I. INTRODUCTION

The Screenless Smart Wristband is a senior graduation project at EDABK Lab, HUST, built around a Nordic nRF52840 microcontroller and a GNSS positioning module. Because the device has no display, all outdoor-positioning debugging during integration and field testing must be done by inspecting the GNSS module's raw serial output rather than by glancing at an on-device readout. This places an unusual amount of weight on log inspection, where any tool that makes the raw log easier to read directly speeds up hardware validation.

The GNSS module used on the wristband, a Quectel LC76G, communicates over UART using the NMEA 0183 protocol, an ASCII sentence-based format originally designed

for marine electronics and now used almost universally by GNSS receivers. A single second of operation can produce a dozen or more interleaved sentences spanning GPS, GLONASS, Galileo, and BeiDou, each with a different field layout. Trying to read this stream by eye to confirm that the fix quality is acceptable or that a reasonable number of satellites are being tracked is genuinely tedious and easy to get wrong.

This report describes a small C program written to address that problem. My program is not part of the wristband firmware itself; it is an offline diagnostic tool that reads a timestamped capture of the module's raw NMEA output and produces a structured, readable report. My contribution, and the scope of this report, is limited to this parsing and validation tool. The wristband hardware and firmware are the work of the EDABK Lab senior project team, and are described here only as context for why the parser was needed.

The remainder of this report is organized as follows. Section II reviews the NMEA 0183 protocol, GNSS constellations, and the LC76G module. Section III describes the parser's design and implementation. Section IV evaluates the parser on a real field log captured from the wristband's GNSS module. Section V concludes.

## II. BACKGROUND

### A. The NMEA 0183 Protocol

NMEA 0183 is an ASCII, single-talker/multiple-listener serial protocol maintained by the National Marine Electronics Association [1]. Each message, or "sentence," begins with a \$ character, followed by a two-letter talker identifier and a three-letter sentence formatter, followed by comma-delimited data fields, and terminated by an optional checksum field introduced by a \* delimiter. Fields may be empty when data is unavailable (e.g., no fix), so a parser must locate them by counting delimiters rather than by fixed character offsets. Although the protocol predates GPS, its simplicity over a single UART pin made it the de facto standard for GNSS receivers, including embedded modules with no other interface budget.

### B. GNSS Constellations and Talker Identifiers

Modern GNSS receivers can track signals from more than one satellite constellation: GPS (United States), GLONASS (Russia), Galileo (European Union), and BeiDou (China).

I am a high school student at The Dewey Schools Tay Ho (Adventure International Program). This work was carried out as part of a lab-assistant contribution to EDABK Lab beginning April 2026.



Fig. 1. The Screenless Smart Wristband PCB. The Quectel LC76G GNSS module is visible on the right (Quectel logo); the u-blox LEXI-R10801D LTE module occupies the center; the main MCU is mounted at the top.

NMEA 0183 identifies the source constellation of a sentence through its two-letter talker ID: GP for GPS, GL for GLONASS, GA for Galileo, and GB for BeiDou. A receiver that fuses observations from multiple constellations into a single position solution typically labels that combined output with the talker ID GN. Multi-constellation operation generally improves time-to-first-fix and positioning accuracy, particularly in partially obstructed sky views, which matters for a wrist-worn device that will be used outdoors under varying conditions.

### C. The Quectel LC76G GNSS Module

The Quectel LC76G is a compact, low-power GNSS module capable of concurrently tracking GPS, GLONASS, BeiDou, Galileo, and QZSS signals [2], [3]. Its small footprint and low power draw make it a common choice for battery-powered wearables, which is why it was selected for the wristband. The module outputs standard NMEA 0183 sentences over UART, including the six sentence types handled by the parser later described in this report.

## III. SYSTEM DESIGN AND IMPLEMENTATION

### A. Pipeline Overview

The parser is implemented as a single-pass C program with six logical stages, shown in Fig. 2. A raw log file is read line by line; each line is checked for a leading timestamp and an NMEA sentence; the sentence is tokenized into fields; the three-letter sentence formatter is used to dispatch to a per-type handler; the handler decodes and writes a structured block to the output report; and a set of running counters is updated for the final summary table.

### B. Timestamp and Sentence Extraction

Each line in the captured log is prefixed with a bracketed timestamp, [YYYY-MM-DD\_HH:MM:SS:mmm], immediately followed by the NMEA sentence. The function `extract_parts` copies the bracketed timestamp into a separate buffer and returns a pointer to the `$` character that begins the sentence. If a line begins directly with `$` (no timestamp), it is still handled correctly. If no `$` character is found at all, the function returns NULL and the caller writes the line verbatim under a “non-NMEA” label rather than guessing at its structure. This fallback path is what allows the program to tolerate corrupted or non-sentence lines, discussed further in Section IV.

### C. Tokenization

NMEA fields are separated by commas, and the sentence ends at a checksum delimiter, `*`, after which the remaining characters are a hexadecimal checksum rather than data. The tokenizer, `split_csv`, scans for the nearer of the next comma or `*` on each iteration, copies the field into a fixed 64-byte buffer with explicit null-termination, and stops as soon as a `*` is reached. The checksum itself is never validated; `*` is used only as a tokenization boundary, so a sentence with a corrupted body but an intact structure would still be parsed and reported like normal. Each sentence handler allocates a fields array sized for the maximum field count of that sentence type (e.g., 24 for GSV, which can carry up to four full satellite records per message), which bounds stack usage per call.

### D. Coordinate Conversion

NMEA latitude and longitude are encoded as `ddmm.mmmm` (degrees followed directly by minutes), rather than decimal degrees. The conversion function `nmea_to_decimal` recovers decimal degrees as

$$\text{decimal} = \left\lfloor \frac{\text{raw}}{100} \right\rfloor + \frac{\text{raw mod } 100}{60}, \quad (1)$$

and negates the result when the hemisphere field is S or W. For example, a raw GGA field of 2055.281683 N is converted to  $20 + 55.281683/60 = 20.921361^\circ\text{N}$ , matching the field as logged.

### E. Talker ID to Constellation Mapping

The function `constellation_name` maps the two-letter talker ID prefix of a sentence (`$GP`, `$GL`, `$GA`, `$GB`, `$GN`) to a human-readable constellation name (GPS, GLONASS, Galileo, BeiDou, or Multi-constellation), used throughout the structured output so that, for example, a GSV block clearly states which constellation’s satellites it describes.

### F. Per-Sentence-Type Handlers

Six dedicated handler functions decode the sentence types observed in the field log:

- **GGA** – the primary fix sentence: UTC time, latitude/longitude, numeric fix quality (mapped to a label such as “GPS fix” or “RTK fixed”), satellite count, HDOP, altitude, and geoid separation.

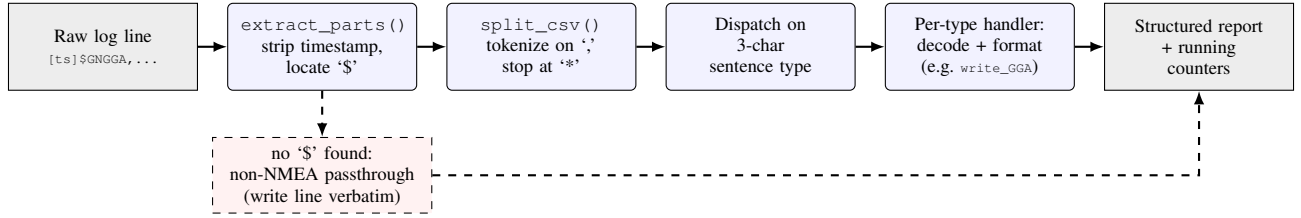


Fig. 2. Parsing pipeline. Each raw log line passes through timestamp/sentence extraction, comma tokenization, type dispatch, and a dedicated decoder before being written to the structured report. Lines with no \$ character bypass tokenization entirely and are written verbatim under a non-NMEA label (dashed path).

- **RMC** – the only standard sentence carrying date along-side time; also reports an active/void status flag, speed (converted from knots to km/h), and course. Position and speed fields are only decoded when the status flag indicates an active fix.
- **GSA** – active satellite PRNs and dilution-of-precision (DOP) values. Each constellation reports its own GSA sentence, so up to four GSA sentences are expected per epoch in a four-constellation receiver.
- **GSV** – satellites in view, in PRN/elevation/azimuth/SNR quadruples. A single epoch’s satellites for one constellation may be split across multiple GSV messages (up to four satellites per message); the handler tracks a running satellite index across messages.
- **VTG** – course over ground and speed, reported in both knots and km/h; redundant with RMC’s speed/course fields but useful as a cross-check.
- **GLL** – a legacy latitude/longitude sentence inherited from older LORAN-era equipment; included for completeness, though it carries strictly less information than GGA.

Every handler begins with a field-count guard: if a sentence has fewer fields than its type requires, the handler writes an explicit “(incomplete sentence, skipped)” note instead of indexing past the end of the tokenized array. Sentences that do not match any of the six known formatters (proprietary \$PAIR/\$PQTM messages, or other vendor-specific output) are written verbatim to the report rather than discarded, so that no information is silently lost.

#### G. Dispatch Loop and Robustness

The main loop reads one line at a time, extracts the timestamp and sentence, and dispatches on the three characters immediately following the two-letter talker ID – a technique that lets the same dispatch logic handle a sentence regardless of which constellation produced it (e.g., \$GPGSV and \$GLGSV both dispatch to `write_GSV`). Seven running counters (one per known sentence type plus one for everything else) are incremented as each line is processed and are printed as a summary table once the file is exhausted. Because all string handling uses fixed-size buffers with explicit length checks and null-termination, malformed input cannot overrun a buffer; at worst, a sentence is reported as incomplete or passed through as unrecognized text.

TABLE I  
NMEA SENTENCE COUNTS, 131-SECOND FIELD CAPTURE

| Type         | Description                         | Count       |
|--------------|-------------------------------------|-------------|
| GGA          | Global positioning fix              | 89          |
| RMC          | Recommended minimum navigation info | 90          |
| GSA          | Active satellites / DOP             | 357         |
| GSV          | Satellites in view                  | 573         |
| VTG          | Course and speed                    | 90          |
| GLL          | Geographic position (legacy)        | 89          |
| Other        | Proprietary / unrecognized          | 12          |
| <b>Total</b> |                                     | <b>1300</b> |

## IV. RESULTS AND EVALUATION

### A. Experimental Setup

The parser was evaluated on a raw NMEA capture taken from the wristband’s LC76G module during a stationary outdoor field test. The capture spans approximately 131 seconds (13:07:44.622 to 13:09:55.359, capture-local time) and contains 1343 raw log lines: 1300 were dispatched as NMEA sentences (1288 decoded into structured fields, 12 proprietary or unrecognized but still well-formed, see Table I), 18 were blank and produced no output, and 25 lacked a leading \$ character entirely and were written verbatim under a non-NMEA label, discussed in Section IV-D.

### B. Sentence-Type Distribution

Table I summarizes the per-type counts produced by the parser’s running counters. GSV dominates the distribution, consistent with the fact that satellites-in-view data for four active constellations (GPS, GLONASS, Galileo, BeiDou) is each split across one or more messages per epoch, while GGA, RMC, VTG, and GLL each appear once per epoch.

### C. Raw-to-Parsed Comparison

Fig. 3 pairs two raw NMEA sentences exactly as captured from the LC76G module with the parser’s corresponding structured output, for a GGA fix and the first message of a GPS GSV block. The reported latitude and longitude in the GGA block match the value obtained by applying Eq. (1) to the raw `ddmm.mmmmm` fields by hand, confirming correct coordinate conversion, and each `Sat #n` entry in the GSV block maps directly onto one repeating PRN/elevation/azimuth/SNR quadruple in the raw, comma-delimited sentence.

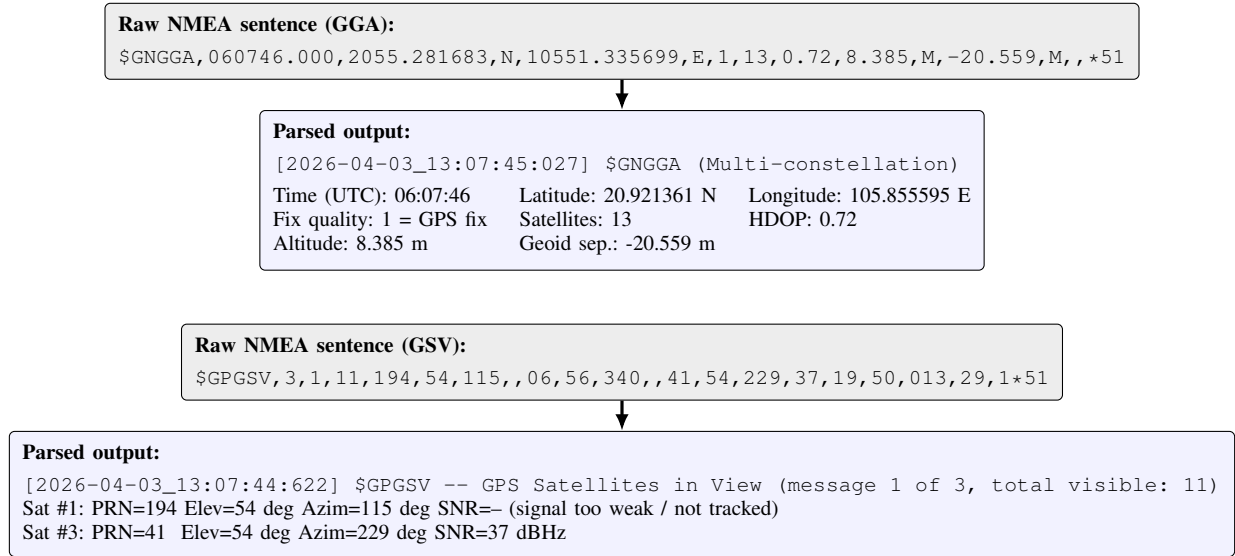


Fig. 3. Raw-to-parsed comparison for a GGA fix sentence and the first message of a GSV satellite-in-view block, both taken directly from the field capture. Each gray box is the literal raw sentence; each blue box is the parser’s corresponding structured output. The GGA fields are arranged in a grid here to save vertical space; the program itself prints one field per line.

#### D. Robustness to Malformed Input

Of the 25 non-NMEA lines, one is a malformed sentence missing its leading `$GNGSA`, formatter, beginning instead with `,A,3,...,1.05,0.72,0.77,3*02`. This was most likely a dropped byte at the module-to-host UART boundary at the very start of the capture – the logger probably attached to the stream a fraction of a second late, cutting off the opening bytes of the first sentence. The remaining 24 were something I did not expect to see: at 13:08:49, the LC76G module’s own bootloader trace appeared mid-stream (22 lines of boot output including `Leaving the BROM`) evidently from an unplanned restart, plus two section markers. All 25 lines fell through to the non-NMEA passthrough path verbatim. The module reboot is worth flagging for the wristband team.”

#### E. Multi-Constellation Behavior

The capture confirms expected multi-constellation structure: GSA sentences appear in groups of up to four per epoch (one per tracked constellation, Section III-F), and GSV blocks for GPS, GLONASS, Galileo, and BeiDou are each present, with GPS contributing the largest share (11 satellites across 3 messages per epoch) and Galileo and BeiDou contributing few or none for most of the capture, consistent with a partial-sky test location. Combined GNRMC, GNVTG, and GNGGA sentences report a single fused position per epoch, with latitude and longitude drifting by under  $0.00002^\circ$  between fixes and altitude varying by a few meters – attributable to normal receiver noise at a stationary point rather than a parsing error, since GLL and RMC independently agree with the corresponding GGA fix.

#### V. CONCLUSION

This report described a small, dependency-free C parser for NMEA 0183 logs, developed to support field validation

of the GNSS subsystem on EDABK Lab’s Screenless Smart Wristband. The parser tokenizes raw sentences, converts coordinates to decimal degrees, maps talker IDs to constellation names, and produces a human-readable report for six core sentence types. I evaluated the parser on a real 131-second, four-constellation field capture containing 1300 parsed sentences; it correctly decoded position, fix-quality, and satellite-visibility data, and passed through every piece of non-NMEA content encountered – a dropped-byte fragment and an unplanned mid-capture reboot of the GNSS module. This tool can be extended with fix-quality alerting thresholds or sentence-timing analysis to further support GNSS debugging on a device with no display. Source code and field-test data are available at <https://github.com/Simbashrek276/NMEA-Data-Parsing-for-Screenless-Smart-Wristband-EDABK-Lab>.

#### ACKNOWLEDGMENTS

This work was carried out in support of the Screenless Smart Wristband project at EDABK Lab, Hanoi University of Science and Technology – a senior graduation project. I thank the EDABK Lab team for the opportunity to contribute the GNSS data-parsing component described in this report.

#### REFERENCES

- [1] National Marine Electronics Association, “NMEA 0183 – Standard for Interfacing Marine Electronic Devices,” Version 4.11, NMEA, Severna Park, MD, USA, 2018.
- [2] Quectel Wireless Solutions Co., Ltd., “LC26G(AB) & LC76G & LC86G Series GNSS Protocol Specification,” GNSS Module Series, Version 1.0.0, 2022.
- [3] Quectel Wireless Solutions Co., Ltd., “LC76G Series Hardware Design,” GNSS Module Series, Version 1.2, 2023.